

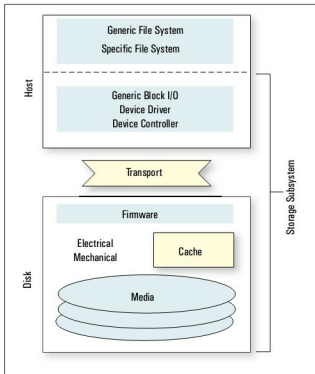


Figure 1: A simplified view of Storage stack

the simple assumption that disks either work or fail, when building reliability mechanisms for file systems. While all-or-nothing disk failures are easier to understand and protect against, the more insidious nature of disk failures is partial failures such as...

- Latent sector errors where a disk block or set of blocks become inaccessible
- Silent disk block corruption
- Disk firmware bugs can result in misdirected or torn disk writes

As data flows from disk through transport (such as the SCSI bus or the network), errors in the transport layer can result in incorrect data being propagated to the host. Next in the chain above are the hardware controller and device driver software present in the host, which are again potential sources of errors. Device drivers can be quite complex and can have bugs which lead to silent data corruption. The next component in the chain is the generic block IO interface of the operating system, which again is quite complex and can contain latent bugs that could result in data corruption.

While these errors are external to the file systems, the file system itself is a complex piece of code of millions of lines, prone to insidious software bugs in its own code, which lead to corruption and inconsistency. Such internal file systems errors are difficult to find and fix during development and can remain latent for a long time, only to rear their heads in production, leading to inconsistent file systems or loss of data to the end user. Given this potential

for error arising in each part of the storage stack, file systems have the enormously complex task of ensuring the highest degree of data availability to the user.

While failures can result in any data block becoming unavailable or inconsistent, the impact of failures of certain blocks that contain critical file system meta-data can result in the entire file system becoming unavailable, resulting in non-availability of vast amounts of data from the end-user perspective. Hence, certain data such as file system meta-data needs to have much higher levels of data integrity than application data. Therefore, the file system needs to have special mechanisms in place for the following reasons:

- To prevent the corruption of meta-data
- In the event of corruption, to be able to detect any such corruption quickly and recover while in operation
- In the worst case event of a crash, achieve a consistent file system state quickly without excessive downtime of data to the users

In common-usage English, the term 'resilience' means the ability to recover quickly from an illness or a misfortune. File systems need to be resilient to errors, no matter where the error occurs in the storage stack. A resilient file system needs to be able to have mechanisms to detect errors or corruptions in meta-data and fix or recover from such errors wherever possible. And in the event of unavoidable crashes, have the ability to return a consistent state without excessive downtime. However, this is not as simple a task as it seems. Detecting faults in file system metadata can be a very complex process, involving the following steps:

0. Make the assumption that the disks are perfect and no errors occur
  1. Check error codes in lower level components in the storage stack below the file system
  2. Perform sanity checks in the file system such as the use of magic numbers and header information for important file system data structures
  3. Check-summing of metadata
  4. Detect failures through metadata replication
- Checksums have been employed widely in storage systems to detect corruption. Check-sums are block hashes computed with a collision-resistant hash function and are used to verify data integrity. For on-disk data integrity, checksums are stored or updated on disk during write operations and read back to verify the block or sector contents during reads.

Many storage systems such as ext4, ZFS, etc, use checksums for on-disk data integrity. Check-summing, if not carefully integrated into the storage system, can fail to protect against complex failures such as lost writes and misdirected writes. While they are typically useful in detecting issues in components below the file system such as the storage